

Лабораторная работа №4

Простая игра

Вариант 11 – Тетрис

ФИО: Ченцов Егор Андреевич, Мильчуков Григорий Викторович

Номер студенческого: 245136, 245141

Группа: ИД24-3

Название предмета: Практикум по программированию

Дата сдачи: 09.12.2025

А. Описание задания

Реализация простой аркадной игры с использованием объектно-ориентированного подхода. Требования включают творческий подход к геймдизайну, визуальному стилю и пользовательскому опыту. Проект должен демонстрировать двумерную анимацию, с возможностью изменения параметров в реальном времени.

В. Описание игры

Тетрис — классическая аркадная игра, разработанная Алексеем Пажитновым в 1984 году. Фигуры (тетромино) падают сверху вниз. Игрок может перемещать их влево/вправо, вращать и ускорять падение. Заполненные горизонтальные линии исчезают, принося очки. Игра заканчивается, если фигуры достигают верхней границы поля.

Достигнутая сложность и реализация

Выполнены все основные требования:

- Добавлено плавное падение фигур
- Предпросмотр следующей фигуры
- Мгновенное падение по пробелу
- Рестарт по клавише R в конце игры

Дополнительно реализовано:

- Параметры (скорость, цвета, размеры) вынесены в config.json для динамических изменений
- Добавил пользовательский хороший интерфейс и предпросмотр
- Использовал цвета, как в классической версии
- Плавная анимация 60 FPS

Используемые инструменты и технологии:

Python 3.10+, библиотека Pygame для графики, ввода и анимации; JSON для конфигурации

С. Распределение ролей и задач

Работа выполнена в паре. Ченцов Егор отвечал за исследование механик игры, разработку базовой версии (логика падения фигур, повороты, очистка линий, счёт очков и коллизии), а также интеграцию конфигурационного файла и основной цикл.

Мильчуков Григорий отвечал за реализацию дополнительного функционала: сохранение рекордов и прогресса в отдельный файл, с загрузкой при запуске, также добавил мультиплеерный режим и провел тестирование полной версии и отладку.

Таким образом, объём задач распределён поровну: базовая механика и логика — Егора часть, расширения и улучшения — часть Григория.

Методы сотрудничества и коммуникации:

Обмен идеями и кодом через Telegram и GitHub (репозиторий проекта). Проводили встречи для обсуждения прогресса и распределения задач.

Распределение задач по созданию графических элементов:

Графические элементы (отрисовка поля, фигур и UI) реализованы Егором в базовой версии. Григорий добавил визуальные эффекты для дополнительного функционала, такие как индикатор рекорда и мультиплеерный режим.

D. Архитектура проекта

Структура ООП

1. **Класс Board.py** — связан с **Tetromino** для проверки коллизий (доска)

- `__init__(self)` — управление сеткой
- `valid_move(self, piece)` — проверка валидности перемещений
- `lock_piece(self, piece)` — фиксацией фигур (интеграция фигуры в доску после падения)
- `lock_piece(self, piece)` — подсчёт очков и линий

2. **Класс Tetromino.py** — самостоятельный класс (фигуры)

- `rotate(self)` — позволяет менять ориентацию фигуры (до 4 состояний)
- `get_positions(self)` — получение позиций клеток фигуры

3. **Класс Renderer.py** — модуль с функциями отрисовки, зависит от Board и Tetromino

- `draw_grid(screen)` — отрисовка сетки поля
- `draw_board(screen, board)` — отрисовка доски и зафиксированных фигур

- `draw_piece(screen, piece)` — динамическая отрисовка падающей фигуры
- `draw_ui(screen, font, board, next_piece)` — отрисовка пользовательского интерфейса

4. **main.py** — загрузка конфига, основной цикл

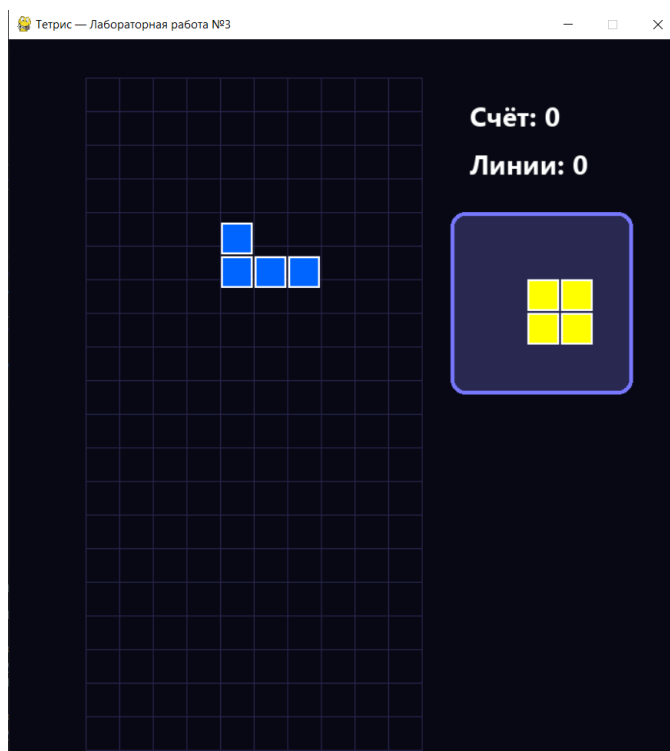
Обоснование использованных шаблонов проектирования:

Мы применили объединение данных и методов для разделения логики игры в классе `Board` и отрисовки в модуле `Renderer`. Вместо наследования использовали композицию, чтобы код был проще и гибче. Настройки вынесли в файл `JSON` для лёгкого изменения без правки основного кода.

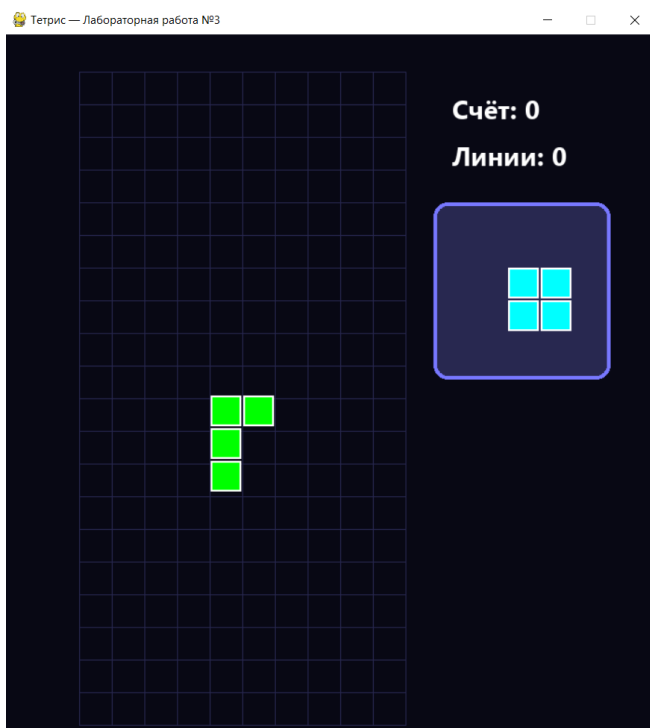
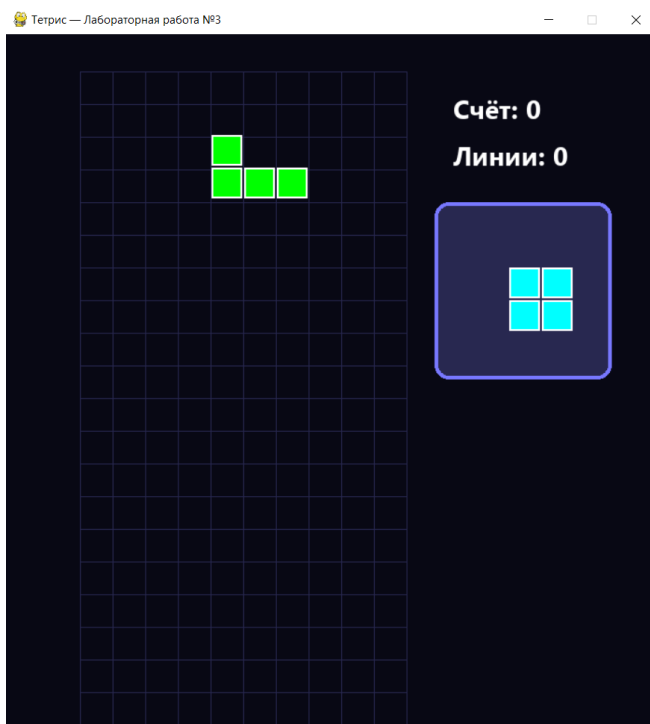
Е. Реализованный функционал

Реализована классическая механика Тетриса с анимацией падения, поворотами, очисткой линий и счётом.

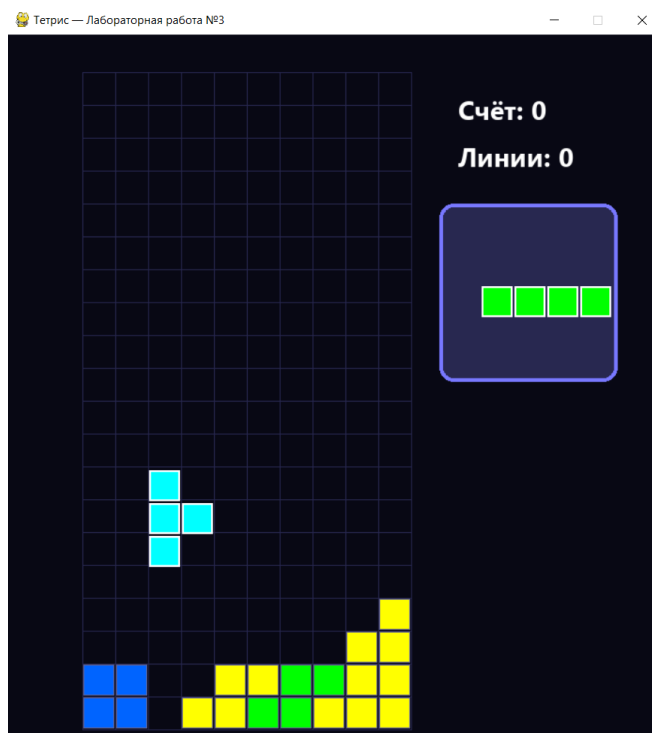
1. Функция: Начало игры с предпросмотром фигуры



2. Функция: Падение и поворот фигуры



3. Функция: Очистка линии и счёт



4. Функция: игра окончена



Фрагменты кода для ключевых алгоритмов

- Пример плавного падения из main.py:

```

151 # Плавное падение с постоянной проверкой коллизии
152 if not board.game_over:
153     # Сохраняем старую позицию
154     old_y_int = current_piece.y
155     old_y_float = piece_y_float
156
157     # Пробуем опуститься
158     piece_y_float += dt / current_speed
159     current_piece.y = int(piece_y_float + 0.999)
160
161     # Проверяем
162     if not board.valid_move(current_piece):
163         # НЕ влезло – возвращаем на предыдущую валидную позицию
164         current_piece.y = old_y_int
165         piece_y_float = float(old_y_int)
166         board.lock_piece(current_piece)
167         current_piece = next_piece
168         next_piece = board.new_piece()
169         piece_y_float = 0.0
170     else:
171         pass # Всё ок – оставляем новую позицию

```

Описание решенных технических проблем

Проблема рывкового падения решена введением дробной координаты Y (piece_y_float) с проверкой коллизий каждый кадр. Проблема с отображением следующей фигуры — центрирование в render.py с расчётом границ.

Г. Инструкции по запуску и игре

Полная схема управления:

- Стрелка влево/вправо: Перемещение фигуры.

- Стрелка вверх: Поворот фигуры.
- Стрелка вниз (удержание): Ускоренное падение.
- Пробел: Мгновенное падение донизу игрового поля.
- R: Рестарт после окончания игры.

Правила и цели игры

Фигуры падают, заполняйте горизонтальные линии для их удаления и набора очков (100 за линию, 300 за две и т.д.). Цель — максимум очков до заполнения поля.

Системные требования и зависимости

Python 3.10+, Pygame (установка: `pip install pygame`). Запуск: `python main.py`

G. Полный исходный код (базовая версия)

(5 файла: `config.json`, `board.py`, `tetromino.py`, `renderer.py`, `main.py`) – базовая часть игры

`config.json`

```
{
  "window_width": 700,
  "window_height": 750,
  "cell_size": 35,
  "cols": 10,
  "rows": 20,
  "background_color": [8, 8, 20],
  "grid_color": [40, 40, 80],
  "fall_speed": 0.5,
  "colors": {
    "I": [0, 255, 255],
    "O": [255, 255, 0],
    "T": [180, 0, 255],
    "S": [0, 255, 0],
    "Z": [255, 0, 0],
    "J": [0, 100, 255],
    "L": [255, 165, 0]
  }
}
```

`board.py`

```
import json

with open("config.json", "r", encoding="utf-8") as f:
    config = json.load(f)

from tetromino import Tetromino
```

```

class Board:
    def __init__(self):
        self.grid = [[0 for _ in range(config["cols"])] for _ in range(config["rows"])]
        self.score = 0
        self.lines = 0
        self.game_over = False

    def valid_move(self, piece):
        for x, y in piece.get_positions():
            if (
                x < 0
                or x >= config["cols"]
                or y >= config["rows"]
                or (y >= 0 and self.grid[y][x])
            ):
                return False
        return True

    def lock_piece(self, piece):
        for x, y in piece.get_positions():
            if y >= 0:
                self.grid[y][x] = piece.color

        # Проверка заполненных строк
        rows_to_clear = []
        for row_idx, row in enumerate(self.grid):
            if all(cell for cell in row):
                rows_to_clear.append(row_idx)

        for row in rows_to_clear:
            del self.grid[row]
            self.grid.insert(0, [0] * config["cols"])

        self.lines += len(rows_to_clear)
        self.score += [0, 100, 300, 500, 800][len(rows_to_clear)]

        # Проверка game over
        if any(self.grid[0]):
            self.game_over = True

    def new_piece(self):
        return Tetromino(config["cols"] // 2 - 1, 0)

```

tetromino.py

import pygame


```

import random
import json

with open("config.json", "r", encoding="utf-8") as f:
    config = json.load(f)

# Все фигуры (в формате 4 поворота)
SHAPES = [
    [[1, 1, 1, 1]], # I
    [[1, 1], [1, 1]], # O
    [[0, 1, 0], [1, 1, 1]], # T
    [[0, 1, 1], [1, 1, 0]], # S
    [[1, 1, 0], [0, 1, 1]], # Z
    [[1, 0, 0], [1, 1, 1]], # J
    [[0, 0, 1], [1, 1, 1]], # L
]

class Tetromino:
    def __init__(self, x, y, shape=None):
        self.x = x
        self.y = y
        self.shape = shape or random.choice(SHAPES)
        self.color = random.choice(list(config["colors"].values()))
        self.rotation = 0

    def rotate(self):
        # Поворот по часовой стрелке
        self.shape = [list(row) for row in zip(*self.shape[::-1])]

    def get_positions(self):
        positions = []
        for row_idx, row in enumerate(self.shape):
            for col_idx, cell in enumerate(row):
                if cell:
                    positions.append((self.x + col_idx, self.y + row_idx))
        return positions

```

renderer.py

```

import pygame
import json

with open("config.json", "r", encoding="utf-8") as f:
    config = json.load(f)

from tetromino import Tetromino

```

```
FIELD_X = 80
FIELD_Y = 40
INFO_X = 480
```

```
def draw_grid(screen):
    w = config["cols"] * config["cell_size"]
    h = config["rows"] * config["cell_size"]

    for i in range(config["cols"] + 1):
        x = FIELD_X + i * config["cell_size"]
        pygame.draw.line(screen, config["grid_color"], (x, FIELD_Y), (x, FIELD_Y + h))
    for i in range(config["rows"] + 1):
        y = FIELD_Y + i * config["cell_size"]
        pygame.draw.line(screen, config["grid_color"], (FIELD_X, y), (FIELD_X + w, y))
```

```
def draw_board(screen, board):
    for y, row in enumerate(board.grid):
        for x, cell in enumerate(row):
            if cell:
                rect = pygame.Rect(
                    FIELD_X + x * config["cell_size"] + 1,
                    FIELD_Y + y * config["cell_size"] + 1,
                    config["cell_size"] - 2,
                    config["cell_size"] - 2,
                )
                pygame.draw.rect(screen, cell, rect)
                pygame.draw.rect(screen, (100, 100, 150), rect, 1)
```

```
def draw_piece(screen, piece):
    for x, y in piece.get_positions():
        if y >= 0:
            rect = pygame.Rect(
                FIELD_X + x * config["cell_size"] + 1,
                FIELD_Y + y * config["cell_size"] + 1,
                config["cell_size"] - 2,
                config["cell_size"] - 2,
            )
            pygame.draw.rect(screen, piece.color, rect)
            pygame.draw.rect(screen, (255, 255, 255), rect, 2)
```

```
def draw_ui(screen, font, board, next_piece):
    # Счёт и линии
    score_text = font.render(f"Счёт: {board.score}", True, (255, 255, 255))
```

```

lines_text = font.render(f"Линии: {board.lines}", True, (255, 255, 255))
screen.blit(score_text, (INFO_X, 60))
screen.blit(lines_text, (INFO_X, 110))

# Рамка следующей фигуры
preview_rect = pygame.Rect(INFO_X - 20, 180, 190, 190)
pygame.draw.rect(screen, (40, 40, 80), preview_rect, border_radius=15)
pygame.draw.rect(screen, (120, 120, 255), preview_rect, 4, border_radius=15)

# по центру
temp = Tetromino(0, 0, next_piece.shape)
temp.color = next_piece.color

# Находим границы фигуры
positions = temp.get_positions()
min_x = min(pos[0] for pos in positions)
max_x = max(pos[0] for pos in positions)
min_y = min(pos[1] for pos in positions)
max_y = max(pos[1] for pos in positions)

width = max_x - min_x + 1
height = max_y - min_y + 1

# Центрируем в рамке
offset_x = INFO_X + 95 - (width * config["cell_size"]) // 2
offset_y = 285 - (height * config["cell_size"]) // 2

for x, y in positions:
    rect = pygame.Rect(
        offset_x + (x - min_x) * config["cell_size"],
        offset_y + (y - min_y) * config["cell_size"],
        config["cell_size"] - 2,
        config["cell_size"] - 2,
    )
    pygame.draw.rect(screen, temp.color, rect)
    pygame.draw.rect(screen, (255, 255, 255), rect, 2)

```

main.py

```

import pygame
import json
import sys
import copy

with open("config.json", "r", encoding="utf-8") as f:
    config = json.load(f)

```

```

from board import Board
from tetromino import Tetromino, SHAPES
from renderer import draw_grid, draw_board, draw_piece, draw_ui

pygame.init()
screen = pygame.display.set_mode((config["window_width"], config["window_height"]))
pygame.display.set_caption("Тетрис — Лабораторная работа №3")
clock = pygame.time.Clock()
font = pygame.font.SysFont("segoeui", 28, bold=True)

board = Board()
current_piece = board.new_piece()
next_piece = board.new_piece()
# плавная версия

piece_y_float = 0.0
base_fall_speed = config["fall_speed"]

running = True
while running:
    dt = clock.tick(60) / 1000.0
    fall_time = dt # используем реальное время кадра

    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            running = False

    if event.type == pygame.KEYDOWN and not board.game_over:
        if event.key == pygame.K_LEFT:
            current_piece.x -= 1
            if not board.valid_move(current_piece):
                current_piece.x += 1
        if event.key == pygame.K_RIGHT:
            current_piece.x += 1
            if not board.valid_move(current_piece):
                current_piece.x -= 1
        if event.key == pygame.K_UP:
            rotated = copy.deepcopy(current_piece)
            rotated.rotate()
            if board.valid_move(rotated):
                current_piece.rotate()
        if event.key == pygame.K_SPACE:
            while board.valid_move(current_piece):
                current_piece.y += 1
                piece_y_float += 1.0
            current_piece.y -= 1
            piece_y_float = current_piece.y

```

```

        board.lock_piece(current_piece)
        current_piece = next_piece
        next_piece = board.new_piece()
        piece_y_float = 0.0

if board.game_over and pygame.key.get_pressed()[pygame.K_r]:
    board = Board()
    current_piece = board.new_piece()
    next_piece = board.new_piece()
    piece_y_float = 0.0

# Определяем скорость падения
current_speed = base_fall_speed
if pygame.key.get_pressed()[pygame.K_DOWN] and not board.game_over:
    current_speed = 0.05

# Плавное падение с постоянной проверкой коллизии
if not board.game_over:
    # Сохраняем старую позицию
    old_y_int = current_piece.y
    old_y_float = piece_y_float

    # Пробуем опуститься
    piece_y_float += dt / current_speed
    current_piece.y = int(piece_y_float + 0.999)

    # Проверяем
    if not board.valid_move(current_piece):
        # НЕ влезло — возвращаем на предыдущую валидную позицию
        current_piece.y = old_y_int
        piece_y_float = float(old_y_int)
        board.lock_piece(current_piece)
        current_piece = next_piece
        next_piece = board.new_piece()
        piece_y_float = 0.0
    else:
        pass # Всё ок — оставляем новую позицию

# Отрисовка
screen.fill(config["background_color"])
draw_grid(screen)
draw_board(screen, board)

# Рисуем с плавным смещением
saved_y = current_piece.y
current_piece.y = piece_y_float
draw_piece(screen, current_piece)

```

```

current_piece.y = saved_y # возвращаем целую координату для логики

draw_ui(screen, font, board, next_piece)

if board.game_over:
    game_over_text = font.render("ИГРА ОКОНЧЕНА", True, (255, 50, 50))
    restart_text = font.render("Нажмите R для рестарта", True, (200, 200, 200))
    screen.blit(game_over_text, (120, 250))
    screen.blit(restart_text, (130, 320))

pygame.display.flip()

pygame.quit()
sys.exit()

```

Дополнительный функционал и изменения базовой версии кода

Сохранение рекордов и прогресса

Это реализовано в классе Board (файл board.py). При создании объекта Board с указанием режима ("single" или "multi") в конструкторе автоматически вызывается load_high_score(), который загружает текущий рекорд для данного режима из файла high_scores.json, а в интерфейсе сразу отображается значение рекорда. Фрагмент:

```

def __init__(self, mode="single"):
    # ...
    self.high_score = self.load_high_score() # загружаем рекорд при старте

def load_high_score(self):
    try:
        with open("high_scores.json", "r") as f:
            data = json.load(f)
            return data.get(self.mode, 0)
    except FileNotFoundError:
        return 0

def save_high_score(self):
    try:
        with open("high_scores.json", "r") as f:
            data = json.load(f)
    except FileNotFoundError:
        data = {}
    data[self.mode] = max(data.get(self.mode, 0), self.score)
    with open("high_scores.json", "w") as f:
        json.dump(data, f)

def lock_piece(self, piece):

```

```
# ... после начисления очков ...
self.score += [0, 100, 300, 500, 800][len(rows_to_clear)]

# ← Вот здесь происходит проверка и сохранение рекорда
if self.score > self.high_score:
    self.high_score = self.score
    self.save_high_score()    # запись в файл сразу при новом рекорде
```

Режим для двух игроков

В main.py при запуске программы игрок выбирает режим, после чего создаётся один (single) или два независимых объекта Board с соответствующими фигурами, аккумуляторами падения и системами управления (WASD+Ctrl для левого игрока и стрелки+Space для правого), что обеспечивает одновременную игру двух человек на одном экране.

Фрагмент, который делает игру двухпользовательской (мультиплеер)

```
# Выбор режима в начале игры
mode = input("Выберите режим: 1 - одиночный, 2 - мультиплеер: ")
multiplayer = mode == "2"
mode_str = "multi" if multiplayer else "single"

# Создание двух игровых полей
board_left = Board(mode=mode_str)
piece_left = board_left.new_piece()
next_piece_left = board_left.new_piece()

board_right = None
piece_right = None
next_piece_right = None
if multiplayer:
    board_right = Board(mode=mode_str)    # второе поле
    piece_right = board_right.new_piece()  # вторая текущая фигура
    next_piece_right = board_right.new_piece() # следующая для второго игрока

В основном цикле — отдельная обработка ввода и падения для каждого игрока

# Левый игрок (WASD + Ctrl)
if not board_left.game_over:
    if event.key == pygame.K_a: # влево
    if event.key == pygame.K_d: # вправо
    if event.key == pygame.K_w: # поворот
    if event.key == pygame.K_LCTRL or event.key == pygame.K_RCTRL:
        board_left.drop_and_lock(piece_left) # мгновенное падение

# Правый игрок (только в мультиплеере, стрелки + Space)
```

```

if multiplayer and board_right and not board_right.game_over:
    if event.key == pygame.K_LEFT:
    if event.key == pygame.K_RIGHT:
    if event.key == pygame.K_UP:
    if event.key == pygame.K_SPACE:
        board_right.drop_and_lock(piece_right)

```

Падение фигур тоже обрабатывается отдельно

```

# Падение левого игрока
if not board_left.game_over:
    speed = 0.05 if keys[pygame.K_s] else base_fall_speed
    fall_accum_left += dt / speed
    # ... движение и lock

# Падение правого игрока
if multiplayer and board_right and not board_right.game_over:
    speed = 0.05 if keys[pygame.K_DOWN] else base_fall_speed
    fall_accum_right += dt / speed
    # ... движение и lock

```

Файл **main.py** (с изменениями)

```

import pygame
import json
import sys
import copy

with open("config.json", "r", encoding="utf-8") as f:
    config = json.load(f)

from board import Board
from tetromino import Tetromino
from renderer import draw_grid, draw_board, draw_piece, draw_ui, FIELD_X_LEFT, FIELD_X_RIGHT,
INFO_X_LEFT, INFO_X_RIGHT

# Выбор режима
mode = input("Выберите режим: 1 - одиночный, 2 - мультиплеер: ")
multiplayer = mode == "2"
mode_str = "multi" if multiplayer else "single"

pygame.init()
screen = pygame.display.set_mode((config["window_width"], config["window_height"]))
pygame.display.set_caption(f"Тетрис — Лабораторная работа №4 | Режим: {'Мультиплеер' if
multiplayer else 'Одиночный'}")
clock = pygame.time.Clock()
font = pygame.font.SysFont("segoeui", 28, bold=True)

```



```

# Игрок слева — WASD (в мультиплеере), справа — стрелки
board_left = Board(mode=mode_str)
piece_left = board_left.new_piece()
next_piece_left = board_left.new_piece()
fall_accum_left = 0.0

board_right = None
piece_right = None
next_piece_right = None
fall_accum_right = 0.0

if multiplayer:
    board_right = Board(mode=mode_str)
    piece_right = board_right.new_piece()
    next_piece_right = board_right.new_piece()

base_fall_speed = config["fall_speed"]
running = True

while running:
    dt = clock.tick(60) / 1000.0
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            running = False

        if event.type == pygame.KEYDOWN:
            # Левый игрок (WASD)
            if not board_left.game_over:
                if event.key == pygame.K_a:
                    piece_left.x -= 1
                    if not board_left.valid_move(piece_left):
                        piece_left.x += 1
                elif event.key == pygame.K_d:
                    piece_left.x += 1
                    if not board_left.valid_move(piece_left):
                        piece_left.x -= 1
                elif event.key == pygame.K_w:
                    rotated = copy.deepcopy(piece_left)
                    rotated.rotate()
                    if board_left.valid_move(rotated):
                        piece_left.rotate()
                elif event.key == pygame.K_LCTRL or event.key == pygame.K_RCTRL:
                    board_left.drop_and_lock(piece_left)
                    piece_left = next_piece_left
                    next_piece_left = board_left.new_piece()
                    fall_accum_left = 0.0

```

```

# Правый игрок (стрелки) — только в мультиплеере
if multiplayer and board_right and not board_right.game_over:
    if event.key == pygame.K_LEFT:
        piece_right.x -= 1
        if not board_right.valid_move(piece_right):
            piece_right.x += 1
    elif event.key == pygame.K_RIGHT:
        piece_right.x += 1
        if not board_right.valid_move(piece_right):
            piece_right.x -= 1
    elif event.key == pygame.K_UP:
        rotated = copy.deepcopy(piece_right)
        rotated.rotate()
        if board_right.valid_move(rotated):
            piece_right.rotate()
    elif event.key == pygame.K_SPACE:
        board_right.drop_and_lock(piece_right)
        piece_right = next_piece_right
        next_piece_right = board_right.new_piece()
        fall_accum_right = 0.0

```

```

keys = pygame.key.get_pressed()

```

```

# Падение левого игрока
if not board_left.game_over:
    speed = 0.05 if keys[pygame.K_s] else base_fall_speed
    fall_accum_left += dt / speed
    old_y = piece_left.y
    piece_left.y = int(fall_accum_left)
    if not board_left.valid_move(piece_left):
        piece_left.y = old_y
        fall_accum_left = old_y
        board_left.lock_piece(piece_left)
        piece_left = next_piece_left
        next_piece_left = board_left.new_piece()
        fall_accum_left = 0.0

```

```

# Падение правого игрока
if multiplayer and board_right and not board_right.game_over:
    speed = 0.05 if keys[pygame.K_DOWN] else base_fall_speed
    fall_accum_right += dt / speed
    old_y = piece_right.y
    piece_right.y = int(fall_accum_right)
    if not board_right.valid_move(piece_right):
        piece_right.y = old_y
        fall_accum_right = old_y

```

```

board_right.lock_piece(piece_right)
piece_right = next_piece_right
next_piece_right = board_right.new_piece()
fall_accum_right = 0.0

# Рестарт по R
if (board_left.game_over and (multiplayer and board_right.game_over or not multiplayer)) or \
    (multiplayer and board_left.game_over and board_right.game_over):
    if keys[pygame.K_r]:
        board_left = Board(mode=mode_str)
        piece_left = board_left.new_piece()
        next_piece_left = board_left.new_piece()
        fall_accum_left = 0.0
        if multiplayer:
            board_right = Board(mode=mode_str)
            piece_right = board_right.new_piece()
            next_piece_right = board_right.new_piece()
            fall_accum_right = 0.0

# Отрисовка
screen.fill(config["background_color"])

# Левое поле (WASD)
draw_grid(screen, FIELD_X_LEFT)
draw_board(screen, board_left, FIELD_X_LEFT)
draw_piece(screen, piece_left, FIELD_X_LEFT)
draw_ui(screen, font, board_left, next_piece_left, INFO_X_LEFT, "1 (WASD)")

if multiplayer:
    # Правое поле (стрелки)
    draw_grid(screen, FIELD_X_RIGHT)
    draw_board(screen, board_right, FIELD_X_RIGHT)
    draw_piece(screen, piece_right, FIELD_X_RIGHT)
    draw_ui(screen, font, board_right, next_piece_right, INFO_X_RIGHT, "2 (Стрелки)")

# Game Over
if board_left.game_over:
    text = font.render("ИГРА ОКОНЧЕНА (Левый игрок)", True, (255, 50, 50))
    screen.blit(text, (FIELD_X_LEFT - 60, 300))
if multiplayer and board_right.game_over:
    text = font.render("ИГРА ОКОНЧЕНА (Правый игрок)", True, (255, 50, 50))
    screen.blit(text, (FIELD_X_RIGHT - 60, 300))

pygame.display.flip()

pygame.quit()
sys.exit()

```

Заключение

В ходе выполнения лабораторной работы была успешно реализована классическая аркадная игра Тетрис с полным соответствием требованиям задания.

Полностью выполнены все обязательные требования: разработана механика падения тетромينو, поворотов, перемещений, очистки заполненных строк, подсчёта очков и проверки окончания игры. Обеспечено управление фигурой с помощью клавиш (стрелки для перемещения и поворота, удержание вниз для ускорения, пробел для мгновенного падения).

Дополнительно реализованы: плавное субпиксельное падение с анимацией 60 FPS, предпросмотр следующей фигуры с центрированием, рестарт по клавише R, вынесение всех параметров в конфигурационный файл config.json для динамических изменений (размеры, цвета, скорость), а также современный интерфейс с закруглёнными элементами и визуальными эффектами.

Был добавлен дополнительный функционал: Режим для двух игроков, а также сохранение рекордов и прогресса

Полученная игра демонстрирует стабильную динамику с учётом физических коллизий и предоставляет пользователю удобный инструмент для экспериментов с параметрами.

Работа выполнена в соответствии с принципами объектно-ориентированного программирования.